

TREINAMENTO DE LLMS

PRÉ-TREINAMENTO, ALINHAMENTO E MÉTODOS DE OTIMIZAÇÃO

INSTRUÇÕES: Cada questão possui exatamente uma alternativa correta.

QUESTÕES

- I) Qual é a função de perda utilizada no pré-treinamento de LLMS (modelagem de linguagem causal)?
- a) Entropia cruzada calculada sobre todos os tokens, incluindo o prompt de instrução.
 - b) Média negativa do logaritmo da probabilidade do próximo token dado o prefixo: $\mathcal{L}_{LM}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t | x_1, \dots, x_{t-1})$.
 - c) Divergência KL entre a distribuição do modelo e uma distribuição de referência fixa.
 - d) Erro quadrático médio entre o embedding do token previsto e o embedding correto.
- II) Segundo a Lei de Chinchilla (Hoffmann et al., 2022), dado um orçamento computacional C fixo, qual é a relação ótima aproximada entre o número de parâmetros N^* e o número de tokens de treinamento D^* ?
- a) $D^* \approx N^*$ (igual número de tokens e parâmetros).
 - b) $D^* \approx 10 \cdot N^*$ (dez tokens por parâmetro).
 - c) $D^* \approx 20 \cdot N^*$ (vinte tokens por parâmetro).
 - d) $D^* \approx 100 \cdot N^*$ (cem tokens por parâmetro).
- III) O GPT-3 possui 175B parâmetros e foi treinado com 300B tokens. De acordo com a Lei de Chinchilla, o que isso indica?
- a) O GPT-3 foi treinado com exatamente o número ótimo de tokens.
 - b) O GPT-3 foi *sub-treinado em dados*: o ótimo de Chinchilla recomendaria aproximadamente 3,5T tokens para 175B parâmetros.
 - c) O GPT-3 foi *super-treinado em dados*: o ótimo de Chinchilla recomendaria apenas 1,75B tokens para 175B parâmetros.
 - d) A Lei de Chinchilla não se aplica ao GPT-3 porque ele usa arquitetura de decoder-only.
- IV) No *Supervised Fine-Tuning* (SFT), por que a perda é calculada apenas sobre os tokens da resposta e não sobre os tokens do prompt/instrução?
- a) Porque os tokens do prompt são tokens especiais que o modelo não consegue processar na retropropagação.
 - b) Para evitar que o modelo otimize para prever as instruções, garantindo que o gradiente venha somente dos tokens que o modelo deve gerar.

- c) Porque calcular a perda sobre o prompt aumentaria o uso de memória de forma impraticável.
 - d) Porque os tokens do prompt já foram aprendidos durante o pré-treinamento e recalculá-los não altera os pesos.
- V) Quantos modelos precisam ser mantidos em memória simultaneamente durante a etapa de otimização com PPO no pipeline RLHF?
- a) 1 modelo (apenas a política sendo treinada).
 - b) 2 modelos (política atual e política de referência SFT).
 - c) 3 modelos (política atual, referência e reward model).
 - d) 4 modelos (política atual, referência SFT, reward model e value function).
- VI) Qual é a principal vantagem do DPO (Rafailov et al., 2023) em relação ao RLHF-PPO do ponto de vista de infraestrutura?
- a) O DPO não requer dados de preferência humana (chosen/rejected).
 - b) O DPO elimina a necessidade de treinar um *Reward Model* separado e de realizar geração *online*, reduzindo significativamente a complexidade de engenharia.
 - c) O DPO é um método *on-policy*, garantindo que os dados de treinamento sejam sempre consistentes com a política atual.
 - d) O DPO permite treinar o modelo sem uma política de referência (π_{ref}).
- VII) O GRPO (*Group Relative Policy Optimization*) substitui a *value function* do PPO por qual mecanismo de estimativa de vantagem?
- a) Uma rede neural auxiliar treinada simultaneamente com a política.
 - b) A média das recompensas de um grupo de respostas geradas para o mesmo prompt, normalizada pelo desvio padrão do grupo (z-score).
 - c) Um reward model fixo que atribui vantagem proporcional ao ranking da resposta.
 - d) A divergência KL entre a política atual e a política de referência.
- VIII) A Lei de Chinchilla indica que o custo computacional de pré-treinamento é aproximadamente $C \approx 6ND$ FLOPs. O que representam N , D e o fator 6 nessa expressão?
- a) N = número de camadas, D = dimensão de embedding, fator 6 = número de operações por camada.
 - b) N = número de parâmetros, D = número de tokens, fator 6 = operações aproximadas por parâmetro por token (forward + backward).
 - c) N = tamanho do vocabulário, D = comprimento máximo de sequência, fator 6 = número de cabeças de atenção.
 - d) N = número de amostras, D = dimensão de embedding, fator 6 = número de etapas de otimização por época.
- IX) Na derivação do DPO, o insight central é que a função de partição $Z(x)$ cancela ao substituir a solução ótima do RLHF no modelo de Bradley-Terry. Por que esse cancelamento ocorre?
- a) Porque $Z(x)$ é sempre igual a 1 quando a política é normalizada corretamente.
 - b) Porque o modelo de Bradley-Terry usa a *diferença* de rewards $r^*(y_w) - r^*(y_l)$, e $Z(x)$ aparece como uma constante aditiva em $r^*(x, y) = \beta \log(\pi^*(y|x)/\pi_{\text{ref}}(y|x)) + \beta \log Z(x)$, cancelando-se na diferença.

- c) Porque $Z(x)$ é aproximado como zero para sequências longas, tornando seu efeito desprezível.
 - d) Porque o DPO usa uma função de ativação sigmoid que satura a diferença de recompensas, eliminando a dependência de $Z(x)$.
- X) O DPO é descrito como um método *off-policy*. Qual é a implicação prática dessa característica?
- a) O DPO não consegue aprender de dados gerados por qualquer modelo diferente do modelo atual.
 - b) O DPO usa dados de preferência coletados com π_{ref} ; se a política π_θ se afastar muito de π_{ref} durante o treinamento, os dados podem não ser mais representativos da distribuição atual.
 - c) O DPO gera novas respostas a cada iteração para garantir que os dados sejam sempre atualizados.
 - d) O DPO ignora completamente a política de referência após a inicialização dos pesos.
- XI) No escalonamento *cosine decay* do LR com fórmula $\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \left(1 + \cos \frac{\pi t}{T}\right)$, qual é o valor do LR em $t = 0$ e em $t = T$?
- a) $\eta_0 = 0$ e $\eta_T = \eta_{\max}$.
 - b) $\eta_0 = \eta_{\max}$ e $\eta_T = \eta_{\min}$.
 - c) $\eta_0 = (\eta_{\max} + \eta_{\min})/2$ e $\eta_T = (\eta_{\max} - \eta_{\min})/2$.
 - d) $\eta_0 = \eta_{\min}$ e $\eta_T = \eta_{\max}$.
- XII) A penalidade KL no objetivo PPO é implementada na prática como um bônus por token. Para cada token t gerado, qual quantidade é subtraída da recompensa?
- a) $\beta \cdot \pi_\theta(t) / \pi_{\text{ref}}(t)$ (razão de probabilidades).
 - b) $\beta \log(\pi_\theta(t) / \pi_{\text{ref}}(t))$ (log da razão de probabilidades).
 - c) $\beta \cdot (\pi_\theta(t) - \pi_{\text{ref}}(t))^2$ (diferença quadrática de probabilidades).
 - d) β/T (constante dividida pelo comprimento da sequência).
- XIII) Por que o GRPO é especialmente adequado para tarefas de raciocínio matemático com reward esparsos?
- a) Porque o GRPO usa um reward model neural especializado em verificação matemática.
 - b) Porque a estimativa de vantagem por grupo (z-score das recompensas do grupo) funciona bem quando o reward é binário (correto/ incorreto) recebido apenas ao final da geração, dispensando a estimativa de valor por estado intermediário.
 - c) Porque o GRPO treina simultaneamente a política e uma value function especializada em passos de raciocínio.
 - d) Porque o GRPO penaliza qualquer resposta que não use *chain-of-thought* explícito.
- XIV) No objetivo *clip surrogate* do PPO, $L^{\text{CLIP}} = \mathbb{E}_t[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)]$, qual é o propósito do operador de *clip*?
- a) Garantir que a razão de importância $r_t = \pi_\theta / \pi_{\text{old}}$ permaneça em $[1-\epsilon, 1+\epsilon]$, impedindo atualizações excessivamente grandes que afastariam muito π_θ de π_{old} .

- b) Normalizar os valores de vantagem A_t para o intervalo $[-1, 1]$.
 - c) Limitar a taxa de aprendizado efetiva para evitar explosão de gradiente.
 - d) Garantir que o reward model nunca atribua recompensas negativas à política.
- XV) O fenômeno de *reward hacking* no RLHF-PPO ocorre quando:
- a) O reward model é treinado com dados insuficientes e começa a atribuir recompensas aleatórias.
 - b) A política aprende a explorar imperfeições do reward model para obter recompensas altas sem melhorar efetivamente a qualidade das respostas (ex.: gerar respostas longas com repetição quando o RM favorece comprimento).
 - c) O otimizador AdamW diverge durante o PPO devido a uma taxa de aprendizado muito alta.
 - d) A penalidade KL se torna muito grande, forçando a política a ser idêntica ao modelo SFT.
- XVI) Qual dos três métodos — PPO, DPO e GRPO — **não** realiza geração *online* durante o treinamento (i.e., não gera novas amostras da política atual a cada iteração)?
- a) PPO.
 - b) DPO.
 - c) GRPO.
 - d) Todos os três realizam geração *online*.
- XVII) Por que, na prática, modelos como o LLaMA são treinados com *mais tokens do que o ótimo de Chinchilla* para o orçamento computacional de treinamento?
- a) Porque a Lei de Chinchilla subestima sistematicamente o número de parâmetros necessário para modelos acima de 7B.
 - b) Porque o treinamento ocorre uma vez, mas a inferência ocorre bilhões de vezes; um modelo menor treinado por mais tokens é mais eficiente na inferência com o mesmo orçamento total de implantação.
 - c) Porque os datasets modernos (ex.: FineWeb) são tão grandes que seria impossível usar apenas 20 tokens por parâmetro.
 - d) Porque treinar com mais tokens do que o ótimo melhora automaticamente a capacidade de *in-context learning*.

XVIII) A perda do DPO é:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right].$$

Qual é o efeito do coeficiente β nessa expressão?

- a) β controla a taxa de aprendizado efetiva; valores maiores de β aumentam a magnitude das atualizações de gradiente.
- b) β controla a temperatura da penalidade KL implícita; valores maiores forçam a política a permanecer mais próxima de π_{ref} , enquanto valores menores permitem maior desvio.
- c) β normaliza as log-probabilidades pelo comprimento das sequências y_w e y_l .
- d) β determina a proporção de pares (chosen, rejected) usados em cada mini-batch de treinamento.

XIX) 🦴 A solução ótima do RLHF com restrição KL é:

$$\pi^*(y|x) = \frac{\pi_{\text{ref}}(y|x) e^{r^*(x,y)/\beta}}{Z(x)}, \quad Z(x) = \sum_y \pi_{\text{ref}}(y|x) e^{r^*(x,y)/\beta}.$$

Isolando $r^*(x, y)$ dessa expressão e substituindo no modelo de preferência de Bradley-Terry $P(y_w > y_l|x) = \sigma(r^*(x, y_w) - r^*(x, y_l))$, obtemos a perda do DPO. Qual afirmação descreve *corretamente* a etapa em que $Z(x)$ é eliminado?

- a) $Z(x)$ é eliminado porque $\sum_y \pi^*(y|x) = 1$ implica $Z(x) = 0$.
- b) Isolando $r^*(x, y) = \beta \log(\pi^*(y|x)/\pi_{\text{ref}}(y|x)) + \beta \log Z(x)$, ao calcular $r^*(x, y_w) - r^*(x, y_l)$ o termo $\beta \log Z(x)$ aparece em ambos com o mesmo sinal e cancela, eliminando $Z(x)$ da perda.
- c) $Z(x)$ é aproximado como constante igual a 1 para grandes vocabulários, tornando sua contribuição desprezível.
- d) $Z(x)$ cancela porque a função sigmoid é invariante a translações constantes no argumento.

XX) 🦴 Considere os três métodos de alinhamento a seguir e associe cada afirmação à(s) característica(s) *exclusiva(s)* de cada método:

Método A – PPO-RLHF: treina um reward model explícito a partir de comparações humanas e usa PPO com penalidade KL para otimizar a política.

Método B – DPO: reparametriza o reward implicitamente em termos da política e da referência, treinando com gradiente descendente supervisionado sobre pares (chosen, rejected).

Método C – GRPO: gera G respostas por prompt, calcula a vantagem como o z-score das recompensas do grupo, e usa clipping de razão de importância sem value function separada.

Qual alternativa descreve **corretamente** uma diferença entre os três métodos em relação à necessidade de um reward model neural?

- a) Os três métodos precisam de um reward model neural externo.
- b) Apenas o PPO-RLHF (Método A) usa um reward model neural explícito; o DPO (Método B) não usa nenhum reward externo; o GRPO (Método C) usa uma função de reward verificável automaticamente (ex.: checagem de resposta matemática), não um modelo neural separado.
- c) O DPO e o GRPO usam o mesmo reward model; apenas o PPO usa um reward model diferente.
- d) Nenhum dos três métodos precisa de um reward model; todos derivam o sinal de treinamento diretamente dos dados de preferência.

XXI) 🦴 Usando a Lei de Chinchilla ($C \approx 6ND$, $N^* \propto \sqrt{C}$, $D^* \approx 20N^*$), calcule o número ótimo aproximado de tokens D^* para treinar um modelo de 13B parâmetros. Em seguida, determine qual afirmação sobre o LLaMA-2-13B (treinado com $\approx 2T$ tokens) está correta.

- a) $D^* \approx 130B$ tokens; o LLaMA-2-13B foi treinado com exatamente o número ótimo de Chinchilla.
- b) $D^* \approx 260B$ tokens; o LLaMA-2-13B foi treinado com aproximadamente $7,7\times$ mais tokens do que o ótimo de Chinchilla, priorizando eficiência de inferência.
- c) $D^* \approx 2T$ tokens; a Lei de Chinchilla recomenda exatamente $2T$ tokens para modelos de 13B parâmetros.

- d) $D^* \approx 1\text{T}$ tokens; o LLaMA-2-13B foi treinado com o dobro do ótimo, o que indica sub-treinamento em parâmetros.

XXII)  O gradiente da perda do DPO em relação aos parâmetros θ pode ser escrito como:

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}} = -\beta \mathbb{E}[\sigma(-\hat{r}_{\theta}(x, y_w, y_l)) (\nabla_{\theta} \log \pi_{\theta}(y_w|x) - \nabla_{\theta} \log \pi_{\theta}(y_l|x))],$$

onde $\hat{r}_{\theta}(x, y_w, y_l) = \beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)}$ é o reward implícito. Qual é a interpretação do fator $\sigma(-\hat{r}_{\theta})$?

- É um fator de normalização constante que garante que o gradiente tenha norma unitária.
- É um peso adaptativo: quando o modelo *subestima* y_w relativo à referência ($\hat{r}_{\theta}$ pequeno), $\sigma(-\hat{r}_{\theta})$ é próximo de 1 e o gradiente é grande; quando y_w já é fortemente favorecido, $\sigma(-\hat{r}_{\theta}) \approx 0$ e o gradiente é pequeno.
- É um fator que garante que o DPO nunca aumente a probabilidade de y_l , independentemente do valor de $\hat{r}_{\theta}$.
- É equivalente à vantagem A_t do PPO, permitindo comparação direta entre os dois métodos.

XXIII)  Em uma rodada de GRPO, um prompt é respondido por $G = 5$ amostras com recompensas $\{r_1, r_2, r_3, r_4, r_5\} = \{1, 0, 1, 0, 1\}$. Calcule a vantagem normalizada $\hat{A}_i = (r_i - \bar{r})/\sigma_r$ para a amostra $i = 1$ (com $r_1 = 1$), dado que $\bar{r} = 0,6$ e $\sigma_r \approx 0,4899$.

- $\hat{A}_1 \approx +0,816$
- $\hat{A}_1 \approx -0,816$
- $\hat{A}_1 \approx +2,041$
- $\hat{A}_1 \approx 0,000$

XXIV)  O SimPO usa normalização por comprimento $\frac{\beta}{|y|} \log \pi_{\theta}(y|x)$ em vez de $\beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$ do DPO. Qual viés o SimPO elimina com essa normalização?

- O viés de posição: tokens no início da sequência recebem gradientes maiores do que tokens no final.
- O viés de comprimento: sem normalização, $\log \pi_{\theta}(y|x)$ é uma soma sobre os tokens e cresce em magnitude com $|y|$, podendo fazer o modelo preferir sistematicamente respostas mais longas (ou mais curtas) apenas por seu comprimento, independentemente da qualidade.
- O viés de vocabulário: tokens mais frequentes recebem log-probabilidades maiores e dominam a perda sem normalização.
- O viés de temperatura: probabilidades estimadas com temperatura $T > 1$ são sistematicamente menores do que as probabilidades reais.

GABARITO

I, Resposta:(b)

O pré-treinamento de LLMs minimiza a perda de modelagem de linguagem causal, que é a média negativa do log da probabilidade do próximo token dado o prefixo:

$$\mathcal{L}_{\text{LM}}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t | x_1, \dots, x_{t-1}).$$

Essa perda é equivalente à entropia cruzada entre a distribuição real (token correto) e a distribuição prevista pelo modelo. A divisão por T normaliza pelo comprimento da sequência, permitindo comparar perdas entre sequências de tamanhos diferentes. A alternativa (a) inclui o prompt, o que não é o caso do pré-treinamento padrão; (c) e (d) descrevem perdas de outros objetivos.

II, Resposta:(c)

Hoffmann et al. (2022) demonstraram que, sob um orçamento computacional fixo $C \approx 6ND$, o ponto de mínima perda é atingido quando $N^* \propto \sqrt{C}$ e $D^* \propto \sqrt{C}$, implicando $D^* \approx 20N^*$. Ou seja, para cada parâmetro do modelo, devem ser vistos aproximadamente 20 tokens de treinamento.

III, Resposta:(b)

Pelo ótimo de Chinchilla, um modelo de 175B parâmetros deveria ser treinado com $D^* \approx 20 \times 175B = 3,5T$ tokens. O GPT-3 utilizou apenas 300B tokens, ou seja, foi *sub-treinado em dados*: o modelo é grande demais para o volume de dados utilizado, consumindo mais computação de treinamento do que o ideal para a qualidade obtida. PaLM (540B, 780B tokens) apresenta a mesma discrepância.

IV, Resposta:(b)

No SFT, mascarar os tokens do prompt (zerar o gradiente para eles) é fundamental para que o modelo aprenda a gerar boas respostas, e não a “prever instruções”. Se a perda fosse calculada também sobre o prompt, os pesos seriam atualizados para maximizar a probabilidade do texto da instrução, o que é irrelevante para o comportamento desejado. A alternativa (d) está errada: mesmo que o pré-treinamento tenha visto texto semelhante ao prompt, o cálculo de gradiente sobre os tokens do prompt ainda afetaria os pesos negativamente.

V, Resposta:(d)

O pipeline RLHF-PPO requer quatro modelos em memória:

1. A política sendo treinada (π_θ).
2. A política de referência congelada (π_{ref} , modelo SFT).
3. O Reward Model (r_ϕ , congelado durante o PPO).
4. A *value function* (crítico, tipicamente do mesmo porte do modelo base).

Esse requisito de $\sim 4 \times$ a VRAM do modelo base é uma das principais motivações para o desenvolvimento do DPO e do GRPO.

VI, Resposta:(b)

O DPO elimina os dois componentes de maior custo do RLHF-PPO: (1) o Reward Model separado, que exige coleta de dados, treinamento e manutenção; e (2) a geração *online*, substituída por treinamento supervisionado direto sobre pares estáticos (chosen, rejected). O DPO ainda usa uma política de referência (π_{ref}), portanto a alternativa (d) está incorreta.

VII, Resposta:(b)

O GRPO substitui a value function do PPO por uma estimativa de baseline derivada do grupo de respostas: para cada prompt, geram-se G respostas, calculam-se suas recompensas $\{r_j\}$, e a vantagem de cada resposta i é o z-score do grupo:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\})}{\text{std}(\{r_j\})}.$$

Isso é simples, estável e dispensa a necessidade de uma rede neural auxiliar de mesmo porte do modelo de política.

VIII, Resposta:(b)

Na fórmula $C \approx 6ND$:

- N = número de parâmetros do modelo.
- D = número de tokens de treinamento.
- Fator 6 = aproximação do número de operações de ponto flutuante por parâmetro por token: ≈ 2 no forward pass, ≈ 4 no backward pass (gradientes de pesos e ativações).

Essa aproximação é útil para estimar o custo computacional de treinamento em FLOPs dado o tamanho do modelo e o volume de dados.

IX, Resposta:(b)

O insight central do DPO é que $Z(x)$ cancela na diferença de rewards. Partindo de:

$$r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x),$$

a diferença de rewards entre a resposta escolhida e a rejeitada é:

$$r^*(x, y_w) - r^*(x, y_l) = \beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)},$$

pois $\beta \log Z(x)$ aparece com o mesmo valor em ambos os termos e cancela. Esse cancelamento é o que elimina a necessidade de calcular $Z(x)$, tornando o DPO tratável sem um reward model explícito.

X, Resposta:(b)

O DPO é off-policy porque usa dados de preferência coletados a partir da política de referência π_{ref} (o modelo SFT), não da política atual π_θ . Conforme o treinamento avança e π_θ se afasta de π_{ref} , os dados podem não representar adequadamente a distribuição da política atual, reduzindo a eficácia do sinal de aprendizado. Isso é diferente do PPO, que é on-policy e gera novas amostras a cada iteração.

XI, Resposta:(b)

Substituindo $t = 0$: $\cos(0) = 1$, logo $\eta_0 = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} (1 + 1) = \eta_{\max}$. Substituindo $t = T$: $\cos(\pi) = -1$, logo $\eta_T = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} (1 - 1) = \eta_{\min}$. O LR começa em $\eta_{\max}$ e decai suavemente até $\eta_{\min}$.

XII, Resposta:(b)

A penalidade KL por token é implementada subtraindo $\beta \log(\pi_\theta(t)/\pi_{\text{ref}}(t))$ da recompensa a cada token t gerado. Isso é a divergência KL ponto a ponto: $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}}) = \mathbb{E}_t[\log(\pi_\theta(t)/\pi_{\text{ref}}(t))]$. Quando $\pi_\theta(t) > \pi_{\text{ref}}(t)$, o log é positivo e a recompensa é reduzida, penalizando o desvio da referência.

XIII, Resposta:(b)

Em tarefas de raciocínio matemático, o reward é esparsos: o modelo recebe +1 apenas se a resposta final estiver correta, e 0 caso contrário. A value function do PPO precisaria estimar o valor esperado para cada estado intermediário da geração de texto, o que é difícil nesse cenário esparsos. O GRPO contorna isso usando apenas a recompensa final e normalizando pelo grupo, obtendo um sinal de treinamento estável sem necessidade de estimar valor por token.

XIV, Resposta:(a)

O clipping limita a razão de importância $r_t = \pi_\theta(a_t|s_t)/\pi_{\text{old}}(a_t|s_t)$ ao intervalo $[1 - \epsilon, 1 + \epsilon]$ (tipicamente $\epsilon = 0,2$). Quando $r_t > 1 + \epsilon$, o ganho adicional não é considerado no cálculo do gradiente, impedindo que o passo de atualização afaste demais π_θ de π_{old} . Isso substitui a restrição de região de confiança do TRPO com uma implementação mais simples e eficiente.

XV, Resposta:(b)

O reward hacking ocorre quando a política encontra estratégias que maximizam o reward do modelo (imperfeito) sem melhorar a qualidade real das respostas. Exemplos documentados incluem geração de respostas excessivamente longas (quando o RM as favorece), uso de frases de afirmação genéricas ou estrutura de formatação que o RM associa incorretamente a qualidade. A penalidade KL (coeficiente β) mitiga esse fenômeno, mas requer calibração cuidadosa.

XVI, Resposta:(b)

O DPO não realiza geração online. Ele treina diretamente sobre um dataset estático de pares (prompt, chosen, rejected) coletados previamente, usando gradiente descendente supervisionado. O PPO e o GRPO, em contraste, são métodos online: ambos geram amostras da política atual a cada iteração (PPO por rollout com o ambiente; GRPO gerando G respostas por prompt).

XVII, Resposta:(b)

O insight econômico chave é a assimetria entre treinamento e inferência: o treinamento acontece uma única vez (custo fixo), mas a inferência pode acontecer bilhões de vezes ao longo da vida do modelo (custo variável). Um modelo menor (portanto mais rápido e barato na inferência) treinado por mais tokens pode ter o mesmo desempenho que um modelo maior treinado pelo ótimo de Chinchilla, mas com custo de inferência muito menor. Essa é a motivação do LLaMA-1/2/3 de usar modelos relativamente pequenos com muitos tokens.

XVIII, Resposta:(b)

O parâmetro β na perda do DPO é herdado da restrição KL do problema RLHF original. Na derivação, β aparece como o inverso da temperatura na penalidade KL: $r^*(x, y) = \beta \log(\pi^*(y|x)/\pi_{\text{ref}}(y|x)) + \dots$ Um β alto significa que o reward implícito varia pouco entre políticas próximas de π_{ref} , forçando o modelo a permanecer conservador. Um β baixo permite maior desvio da referência em direção à maximização do reward.

XIX, Resposta:(b)

O cancelamento de $Z(x)$ ocorre na diferença de rewards porque $\beta \log Z(x)$ é idêntico para y_w e y_l (depende apenas de x , não da resposta):

$$\begin{aligned} r^*(x, y_w) - r^*(x, y_l) &= \left[\beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} + \beta \log Z(x) \right] - \left[\beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)} + \beta \log Z(x) \right] \\ &= \beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}. \end{aligned}$$

A alternativa (a) é incorreta: $Z(x)$ não é zero — é a função de partição que normaliza π^* . A alternativa (c) é incorreta: não há aproximação; o cancelamento é exato. A alternativa (d) é incorreta: a sigmoid preserva constantes aditivas no argumento ($\sigma(a+c-b-c) = \sigma(a-b)$ é verdade, mas isso é consequência do cancelamento, não a causa).

XX, Resposta:(b)

A distinção fundamental é:

- **PPO-RLHF**: requer um reward model neural (r_ϕ) treinado explicitamente em dados de comparação humana, usando a perda de Bradley-Terry.
- **DPO**: não usa nenhum reward externo. O sinal de treinamento vem diretamente dos dados de preferência via a perda supervisionada sobre $(\log \pi_\theta(y_w|x) / \pi_{\text{ref}}(y_w|x) - \log \pi_\theta(y_l|x) / \pi_{\text{ref}}(y_l|x))$.
- **GRPO**: usa uma função de reward verificável automaticamente (ex.: checar se a resposta matemática está correta com um parser simbólico, ou se o código passa nos testes unitários). Não é um modelo neural separado.

XXI, Resposta:(b)

Pelo ótimo de Chinchilla:

$$D^* \approx 20 \times N^* = 20 \times 13\text{B} = 260\text{B tokens}.$$

O LLaMA-2-13B foi treinado com $\approx 2\text{T}$ tokens, que é aproximadamente $2\text{T}/260\text{B} \approx 7,7 \times$ o ótimo de Chinchilla. Essa decisão foi intencional: ao treinar com muito mais dados, o modelo menor de 13B parâmetros se torna mais eficiente na inferência (menor latência e custo por requisição) do que um modelo maior treinado pelo ótimo de Chinchilla com o mesmo orçamento computacional total de implantação.

XXII, Resposta:(b)

O fator $\sigma(-\hat{r}_\theta)$ é um peso adaptativo derivado da função sigmoid aplicada ao reward implícito negado:

- Se $\hat{r}_\theta \ll 0$ (modelo subestima y_w relativo à referência): $\sigma(-\hat{r}_\theta) \approx 1$ — o gradiente é grande, pois há muito a corrigir.
- Se $\hat{r}_\theta \gg 0$ (modelo já favorece fortemente y_w): $\sigma(-\hat{r}_\theta) \approx 0$ — o gradiente é pequeno, pois a política já está bem calibrada.

Esse mecanismo adapta automaticamente a magnitude do gradiente de acordo com a confiança atual do modelo, similar a como o gradiente de uma log-verossimilhança é ponderado pela probabilidade residual.

XXIII, Resposta:(a)

Dados: $r_1 = 1, \bar{r} = 0,6, \sigma_r \approx 0,4899$.

$$\hat{A}_1 = \frac{r_1 - \bar{r}}{\sigma_r} = \frac{1 - 0,6}{0,4899} = \frac{0,4}{0,4899} \approx 0,816.$$

Para verificação, $\sigma_r = \sqrt{\frac{(1-0,6)^2 \cdot 3 + (0-0,6)^2 \cdot 2}{5}} = \sqrt{\frac{3 \cdot 0,16 + 2 \cdot 0,36}{5}} = \sqrt{\frac{1,2}{5}} = \sqrt{0,24} \approx 0,4899$.
As amostras com $r_i = 0$ teriam $\hat{A}_i = (0 - 0,6)/0,4899 \approx -1,225$.

XXIV, Resposta:(b)

A log-probabilidade de uma sequência é a soma das log-probabilidades de cada token:

$$\log \pi_\theta(y|x) = \sum_{t=1}^{|y|} \log \pi_\theta(y_t | x, y_{<t}).$$

Sem normalização, $\log \pi_\theta(y|x)$ cresce em magnitude com $|y|$, fazendo com que sequências mais longas (ou mais curtas) sejam sistematicamente favorecidas ou penalizadas independentemente de sua qualidade semântica. A normalização por $|y|$ converte a soma numa média por token, removendo essa dependência de comprimento. O SimPO adiciona também uma margem γ para garantir separação mínima entre chosen e rejected.